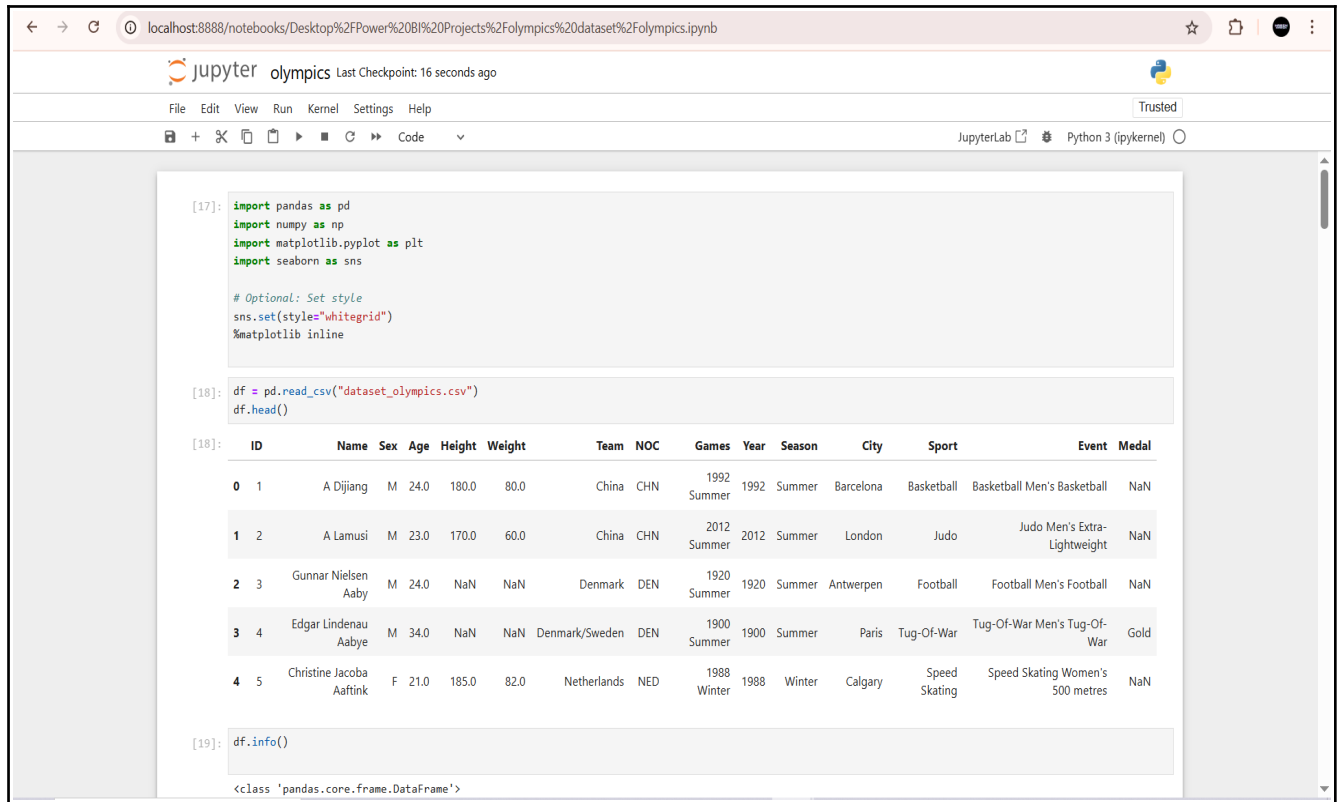


Task 5: Exploratory Data Analysis (EDA)

Objective: Extract insights using visual and statistical exploration.

Tools: Python (Pandas, Matplotlib, Seaborn)

1). Import:



```
[17]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Optional: Set style
sns.set(style="whitegrid")
%matplotlib inline

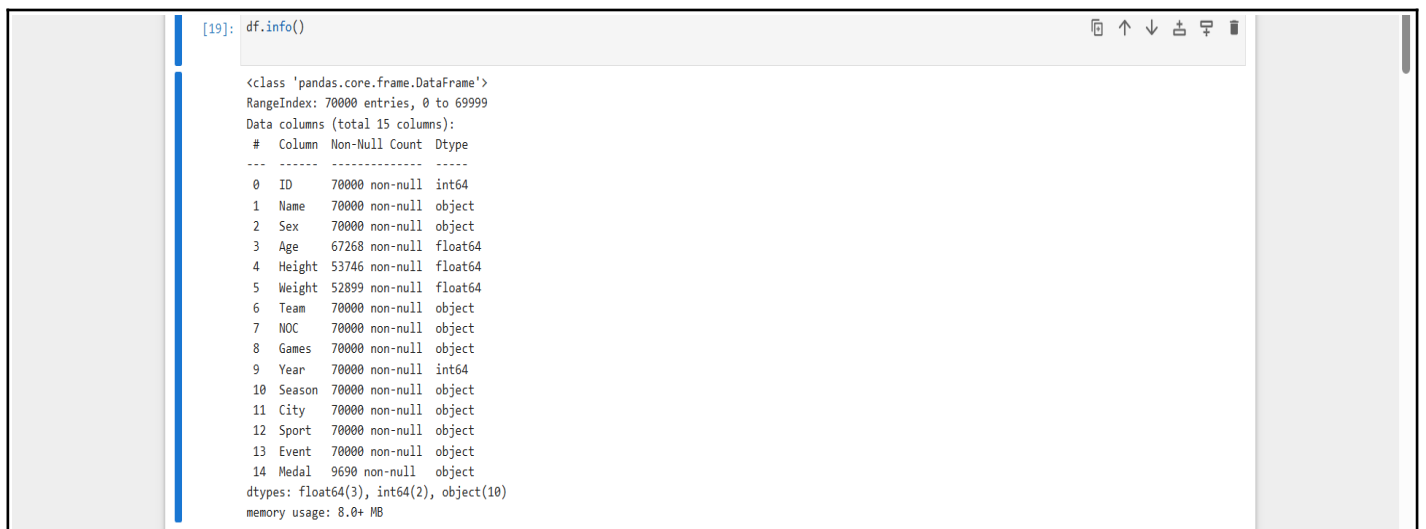
[18]: df = pd.read_csv("dataset_olympics.csv")
df.head()
```

	ID	Name	Sex	Age	Height	Weight	Team	NOC	Games	Year	Season	City	Sport	Event	Medal
0	1	A Dijiang	M	24.0	180.0	80.0	China	CHN	1992 Summer	1992	Summer	Barcelona	Basketball	Basketball Men's Basketball	NaN
1	2	A Lamusi	M	23.0	170.0	60.0	China	CHN	2012 Summer	2012	Summer	London	Judo	Judo Men's Extra-Lightweight	NaN
2	3	Gunnar Nielsen Aaby	M	24.0	NaN	NaN	Denmark	DEN	1920 Summer	1920	Summer	Antwerpen	Football	Football Men's Football	NaN
3	4	Edgar Lindennau Aabye	M	34.0	NaN	NaN	Denmark/Sweden	DEN	1900 Summer	1900	Summer	Paris	Tug-Of-War	Tug-Of-War Men's Tug-Of-War	Gold
4	5	Christine Jacobsa Aaftink	F	21.0	185.0	82.0	Netherlands	NED	1988 Winter	1988	Winter	Calgary	Speed Skating	Speed Skating Women's 500 metres	NaN

```
[19]: df.info()

<class 'pandas.core.frame.DataFrame'>
```

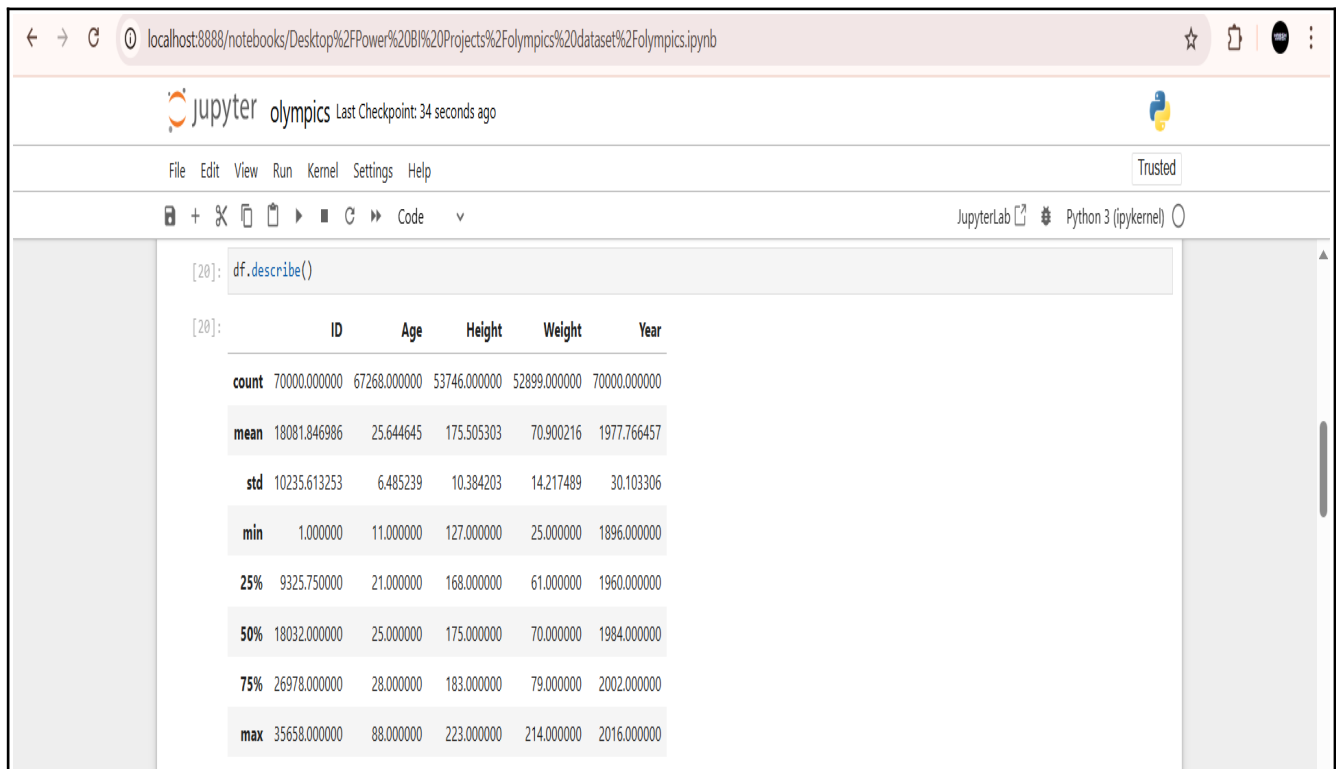
2). info ():



```
[19]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 70000 entries, 0 to 69999
Data columns (total 15 columns):
 #   Column  Non-Null Count  Dtype  
---  --
 0   ID      70000 non-null   int64  
 1   Name    70000 non-null   object  
 2   Sex     70000 non-null   object  
 3   Age     67268 non-null   float64 
 4   Height  53746 non-null   float64 
 5   Weight  52899 non-null   float64 
 6   Team    70000 non-null   object  
 7   NOC     70000 non-null   object  
 8   Games   70000 non-null   object  
 9   Year    70000 non-null   int64  
10  Season  70000 non-null   object  
11  City    70000 non-null   object  
12  Sport   70000 non-null   object  
13  Event   70000 non-null   object  
14  Medal   9690 non-null    object  
dtypes: float64(3), int64(2), object(10)
memory usage: 8.0+ MB
```

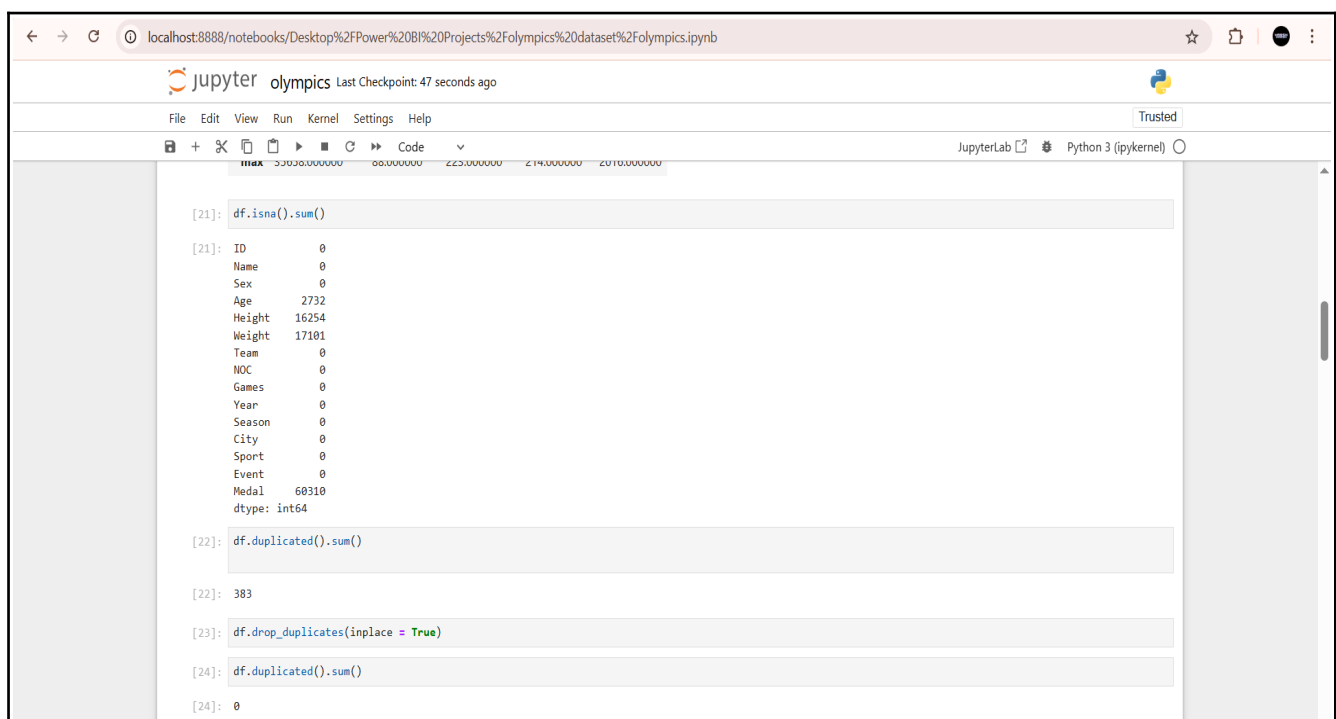
3). Discribe:



The screenshot shows a JupyterLab interface with a browser window at localhost:8888. The notebook is titled 'olympics' and shows the output of the command `df.describe()`. The output is a summary statistics table for the 'olympics' dataset.

	ID	Age	Height	Weight	Year
count	70000.000000	67268.000000	53746.000000	52899.000000	70000.000000
mean	18081.846986	25.644645	175.505303	70.900216	1977.766457
std	10235.613253	6.485239	10.384203	14.217489	30.103306
min	1.000000	11.000000	127.000000	25.000000	1896.000000
25%	9325.750000	21.000000	168.000000	61.000000	1960.000000
50%	18032.000000	25.000000	175.000000	70.000000	1984.000000
75%	26978.000000	28.000000	183.000000	79.000000	2002.000000
max	35658.000000	88.000000	223.000000	214.000000	2016.000000

4). Basic Info and Summary Statistics.



The screenshot shows a JupyterLab interface with a browser window at localhost:8888. The notebook is titled 'olympics' and shows the output of several commands related to data cleaning and summary statistics.

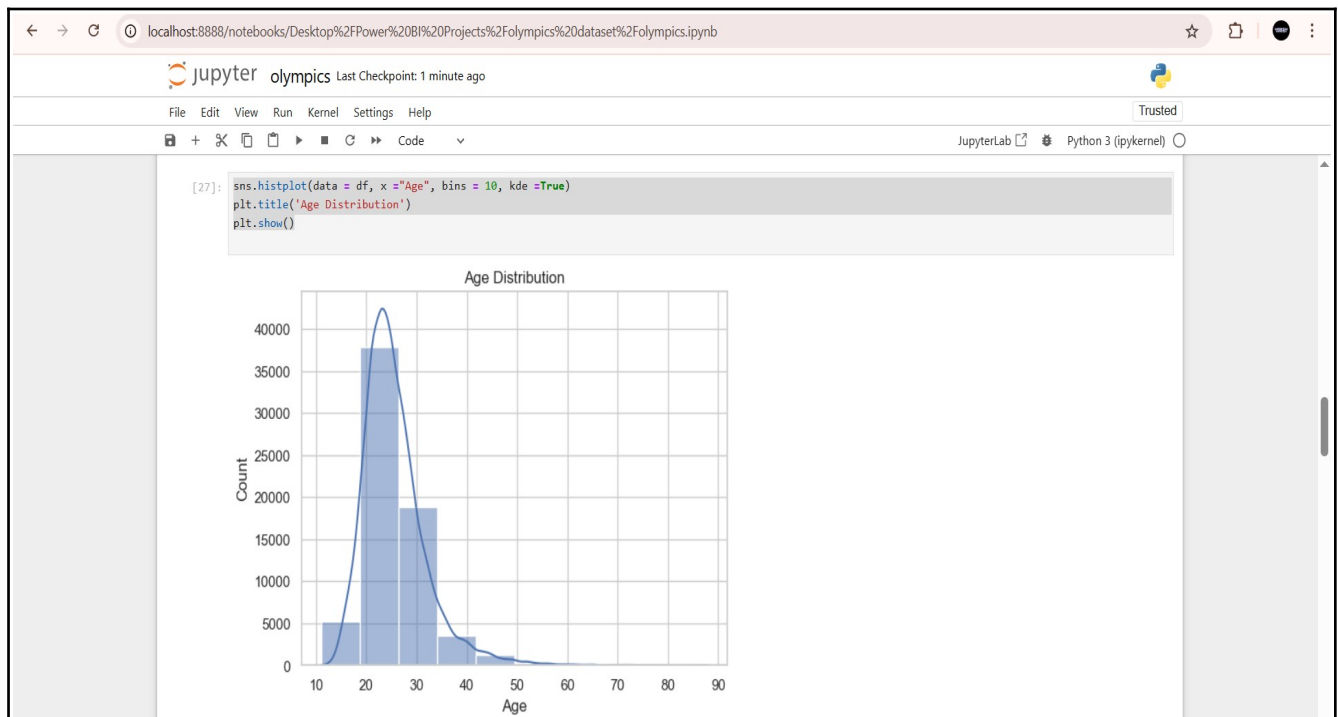
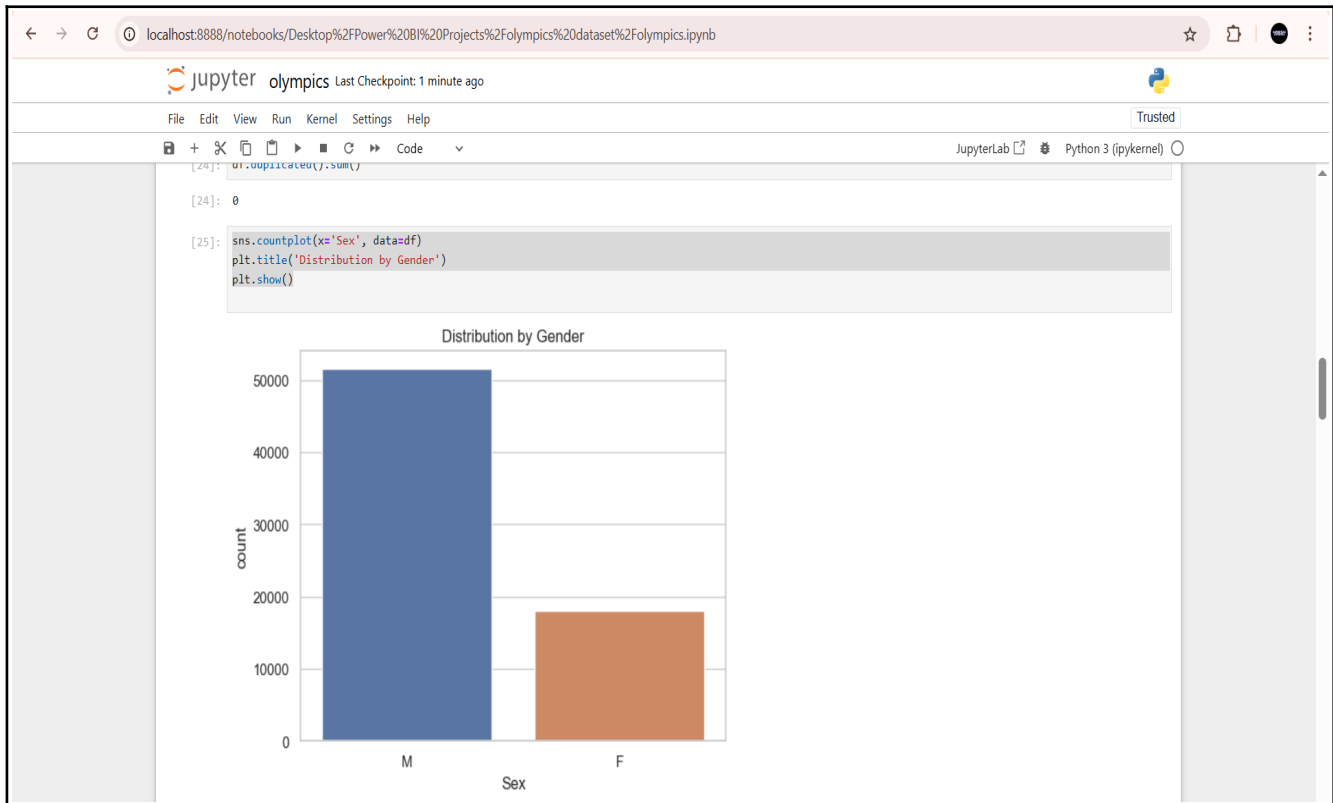
```
[21]: df.isna().sum()
ID      0
Name    0
Sex      0
Age     2732
Height  16254
Weight  17101
Team     0
NOC      0
Games    0
Year     0
Season   0
City     0
Sport    0
Event    0
Medal    60310
dtype: int64

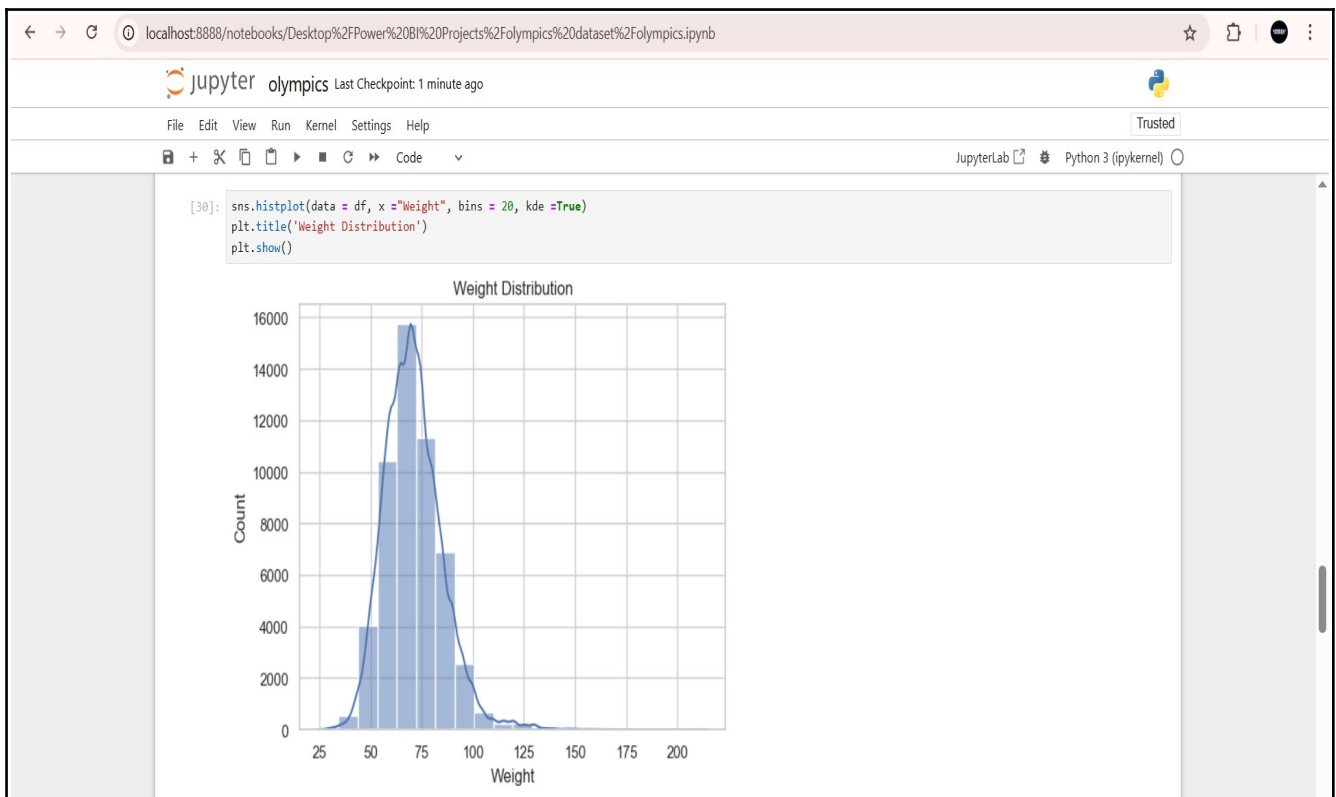
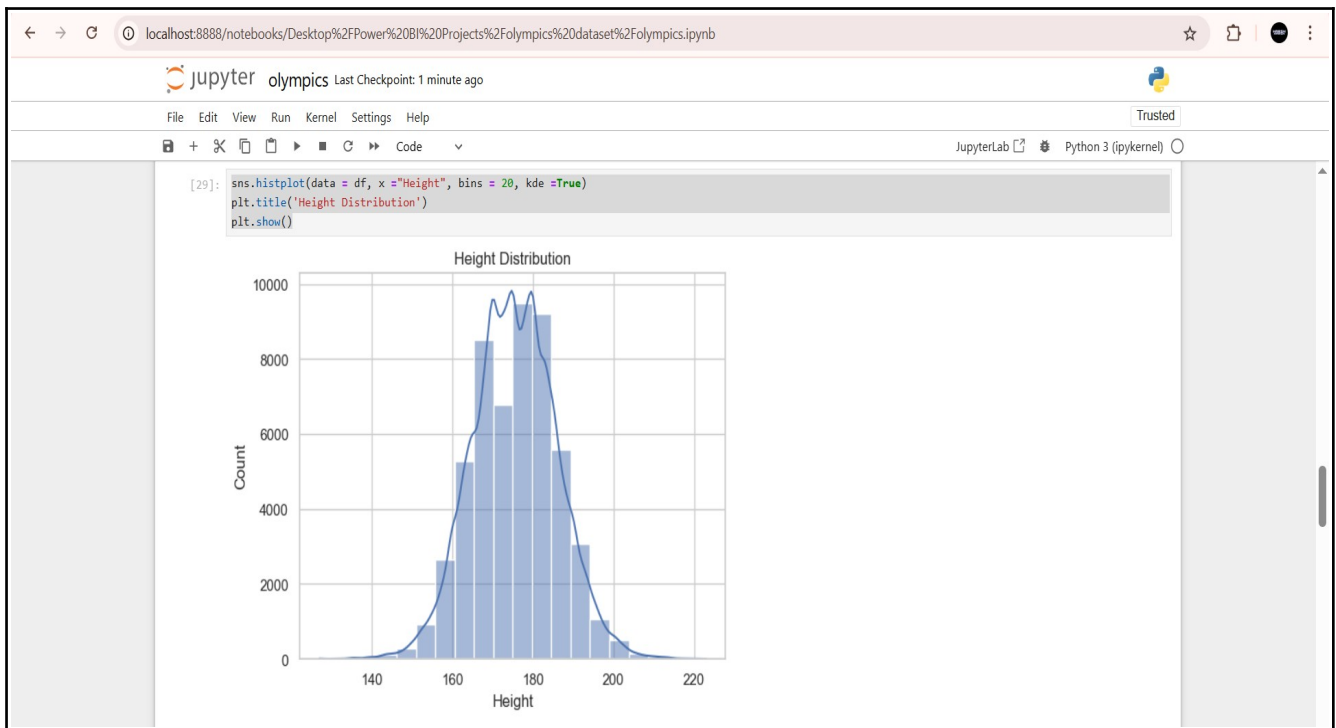
[22]: df.duplicated().sum()
383

[23]: df.drop_duplicates(inplace = True)

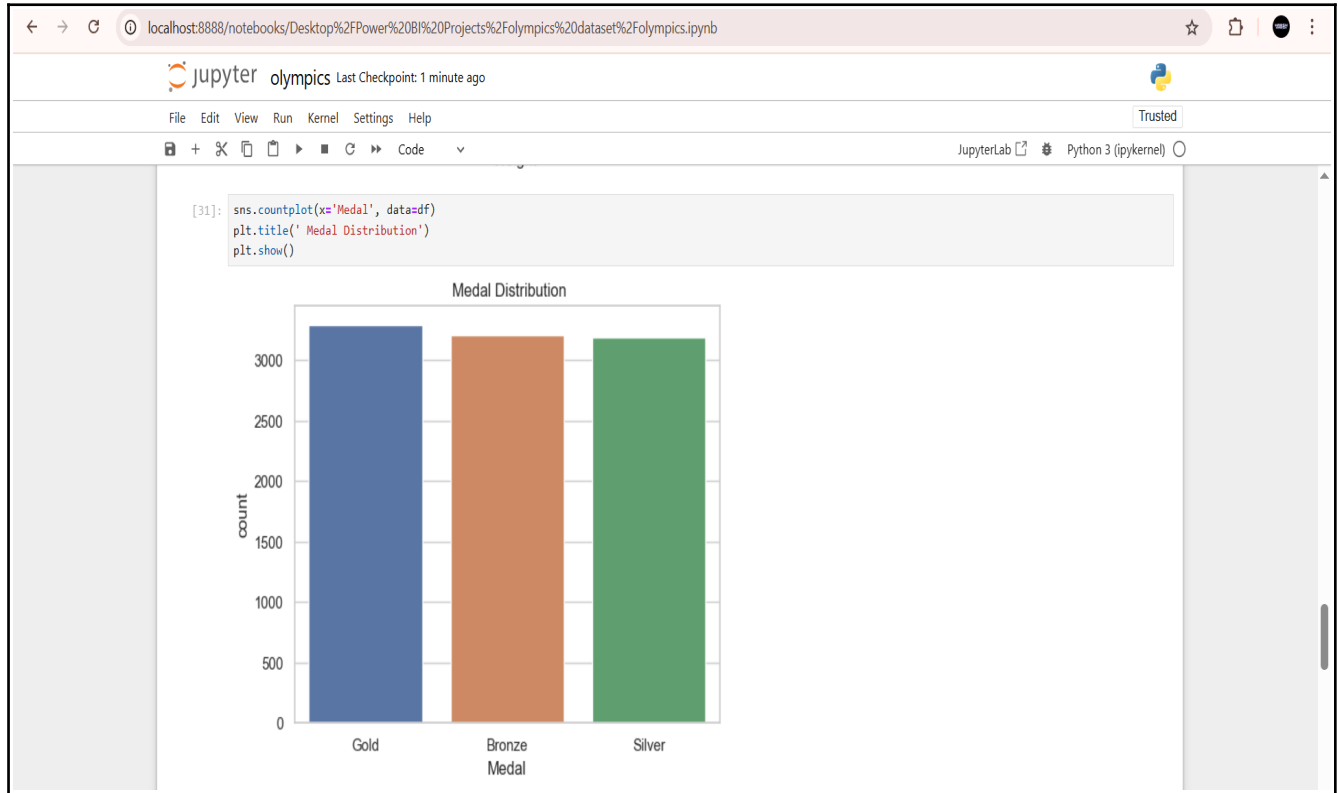
[24]: df.duplicated().sum()
0
```

5). Categorical Features:

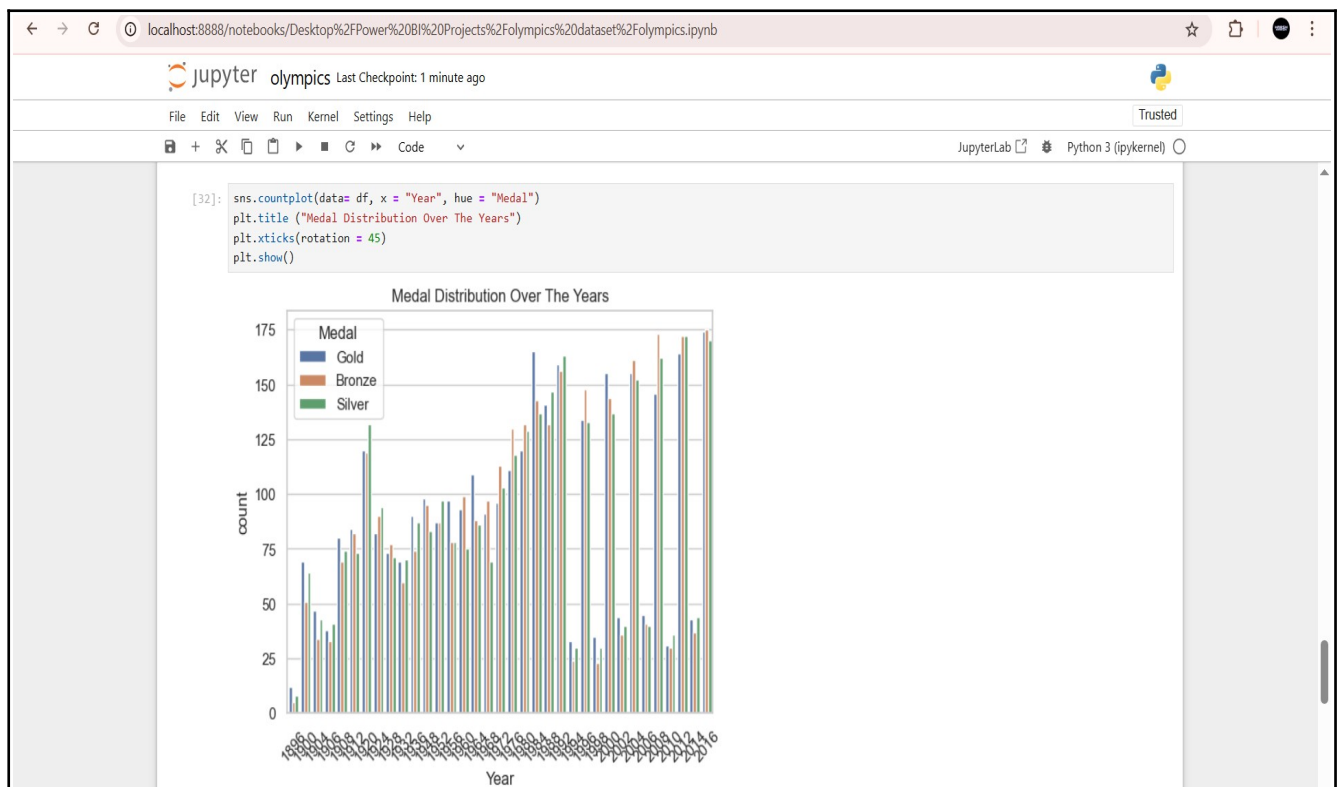




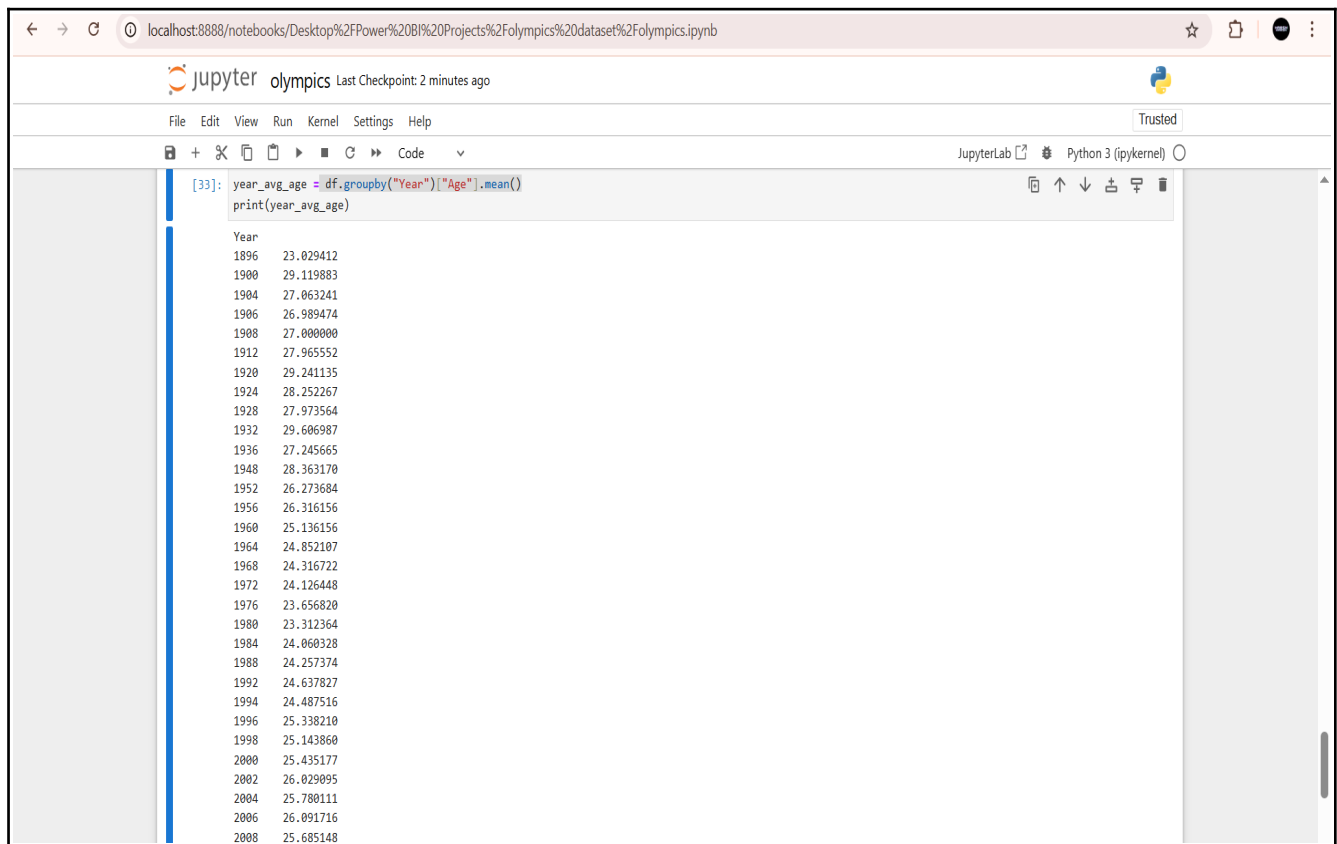
6). Numerical Features:



7). Scatterplots / Pairplots:



8). Avg. of to colomns

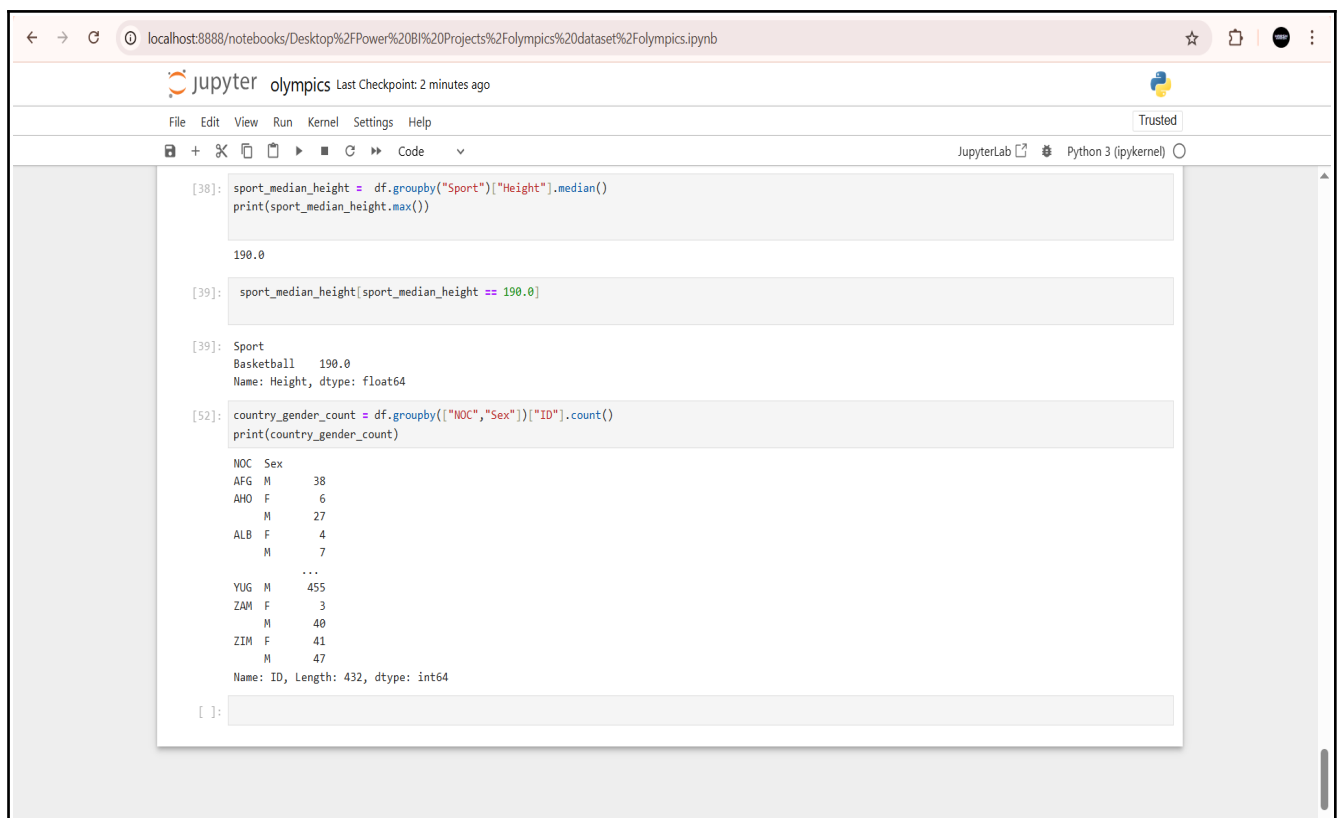


The screenshot shows a JupyterLab interface with a code cell containing the following Python code:

```
[33]: year_avg_age = df.groupby("Year")["Age"].mean()
      print(year_avg_age)
```

The output of the code is a series of mean ages for each year from 1896 to 2008:

Year	Mean Age
1896	23.029412
1900	29.119883
1904	27.063241
1906	26.989474
1908	27.000000
1912	27.965552
1920	29.241135
1924	28.252267
1928	27.973564
1932	29.606987
1936	27.245665
1948	28.363170
1952	26.273684
1956	26.316156
1960	25.136156
1964	24.852107
1968	24.316722
1972	24.126448
1976	23.656820
1980	23.312364
1984	24.060328
1988	24.257374
1992	24.637827
1994	24.487516
1996	25.338210
1998	25.143860
2000	25.435177
2002	26.029095
2004	25.780111
2006	26.091716
2008	25.685148



The screenshot shows a JupyterLab interface with two code cells. The first code cell contains the following Python code:

```
[38]: sport_median_height = df.groupby("Sport")["Height"].median()
      print(sport_median_height.max())
```

The output of the first code cell is:

```
190.0
```

The second code cell contains the following Python code:

```
[39]: sport_median_height[sport_median_height == 190.0]
```

The output of the second code cell is:

```
[39]: Sport
Basketball    190.0
Name: Height, dtype: float64
```

The third code cell contains the following Python code:

```
[52]: country_gender_count = df.groupby(["NOC", "Sex"])["ID"].count()
      print(country_gender_count)
```

The output of the third code cell is:

```
NOC Sex
AFG M    38
AHO F     6
    M    27
ALB F     4
    M     7
...
YUG M   455
ZAM F     3
    M    40
ZIM F    41
    M    47
Name: ID, Length: 432, dtype: int64
```

Outcome: Gain skill in finding patterns, trends, and anomalies